

HOG + LBP + SVM ile Kaş Bölgesi Üzerinden Deepfake Tespiti

Kod ve Deney Açıklama Raporu

Düzeltilmiş Google Colab notebook'unun çalışma mantığı, veri güvenliği, özellik çıkarımı, SVM eğitimi, değerlendirme ve çıktı üretim sürecinin teknik açıklaması

Alan	Değer
Deney yaklaşımı	HOG + LBP özellik füzyonu + SVM
İncelenen bölge	Kaş (Eyebrow ROI)
Önceden eğitilmiş model	Kullanılmıyor
Pozitif sınıf	Fake
Seed	42
Gerçek çalıştırma Run ID	20260807_0948_eyebrow_hog_lbp_svm_seed42

1. Raporun Amacı

Bu rapor, HOG + LBP + SVM tabanlı kaş bölgesi deepfake tespit notebook'unun genel olarak ne yaptığını, her kod bloğunun ne işe yaradığını ve deneyin neden bu şekilde tasarlandığını açıklamak amacıyla hazırlanmıştır. Notebook klasik makine öğrenmesi yaklaşımı kullanır; yani görüntüler doğrudan bir derin öğrenme ağına verilmez. Önce kaş görüntülerinden elle tasarlanmış görsel özellikler çıkarılır, daha sonra bu özellikler bir SVM sınıflandırıcısına öğretilir.

Temel amaç: Kaş ROI'sindeki şekil/yönelim ve mikro-doku bilgisinin, gerçek ve sahte görüntüleri ayırmada tek başına ne kadar yararlı olduğunu ölçmektir.

2. Genel Çalışma Akışı

Kaş ROI görüntüleri → Metadata doğrulama → Train/Validation/Test sızıntı kontrolü → Grayscale → 128×64 resize → HOG çıkarımı + LBP çıkarımı → Özelliklerin birleştirilmesi → StandardScaler (yalnızca train üzerinde öğrenir) → SVM adaylarının validation üzerinde karşılaştırılması → En iyi SVM + karar threshold'unun seçilmesi → Nihai test → Frame-level ve video-level sonuçlar → Model, metrik, tahmin, grafik ve konfigürasyon dosyalarının kaydedilmesi

2.1 Bu yaklaşım neden kullanılıyor?

- HOG, kaşın biçim, kenar ve yönelim bilgisini sayısal hale getirir.
- LBP, kaş kılları ve çevre deri dokusundaki yerel/mikro doku örüntülerini temsil eder.
- Bu iki özellik birleştirilerek hem geometrik hem dokusal bilgi aynı örnek için birlikte kullanılır.
- SVM, bu özellik uzayında Real ve Fake örneklerini ayıran karar sınırını öğrenir.
- Yaklaşım hafif, hızlı ve açıklanabilir bir klasik makine öğrenmesi baseline'ı oluşturur.

3. Notebook Başlangıcı: Bağımlılıklar, Drive ve Konfigürasyon

Notebook ilk aşamada scikit-image, scikit-learn, PyYAML, joblib ve OpenCV gibi ihtiyaç duyulan kütüphaneleri kurar ve Google Drive'ı Colab ortamına bağlar. Tüm önemli parametreler tek bir YAML konfigürasyonunda tutulur; böylece görüntü boyutu, HOG/LBP ayarları, SVM adayları, seed ve klasör yolları kodun farklı yerlerine dağılmaz.

3.1 Düzeltilmiş notebook'taki YAML hata koruması

Düzeltilmiş sürümde config dosyası okunurken YAML biçim hatası oluşursa eğitim doğrudan belirsiz bir şekilde devam etmez. Bozuk config dosyası '.yaml.broken' uzantısıyla yedeklenir, varsayılan ve geçerli konfigürasyon yeniden oluşturulur ve ardından config tekrar okunur. Bu mekanizma, önceki çalıştırmada görülen 'ScannerError: mapping values are not allowed here' hatasının tekrar eğitimi engellemesini önler.

Parametre	Notebook değeri
Görüntü boyutu	128 × 64
HOG orientations	9

HOG pixels_per_cell	8 × 8
HOG cells_per_block	2 × 2
HOG block norm	L2-Hys
LBP points	8
LBP radius	1
LBP method	uniform
SVM kernel adayları	linear, rbf
SVM C adayları	0.1, 1.0, 10.0
RBF gamma adayları	scale, 0.001, 0.0001
Class weight	balanced
Model seçim metriği	Validation F1
Pozitif sınıf	fake

4. Kaynak Veri ve Dosya Güvenliği

Notebook, daha önce kaş bölgesine kırılmış görüntüleri doğrudan giriş olarak kullanır. Kaş klasöründeki görüntüler yeniden kırılmaz, yeniden adlandırılmaz, taşınmaz veya üzerine yazılmaz. Notebook yalnızca bu dosyaları okur; üretilen tüm sonuçlar ayrı bir Run ID klasörüne yazılır.

- Girdi klasörü: Deneyler/Nazlıcan/Deney 1/Kaş
- ROI metadata: Kaş/metadata.csv
- Kaynak frame metadata: Deney 1 Frame/secim_metadata.csv
- Çıktı: Deneyler/Nazlıcan/Deney 1/Sonuçlar/<run_id>/
- Yeni run mevcut run klasörünün üzerine yazılmaz.

4.1 Neden iki metadata dosyası birlikte kullanılıyor?

Kaş ROI metadata'sında kaynak video bilgisinin eksik olabildiği durumda notebook bunu tahmin ederek uydurmaz. Her kaş frame'inin dosya adı, seçim metadata'sındaki gerçek orijinal frame yolu ile birebir eşleştirilir. Kaynak video kimliği ve orijinal frame index'i bu gerçek yoldan geri kazanılır. Bu adım özellikle video bazlı veri sızıntısını kontrol edebilmek için gereklidir.

5. Veri Muhasebesi ve Veri Sızıntısı Kontrolleri

Model eğitilmeden önce metadata şeması doğrulanır. Zorunlu sütunlardan biri eksikse eğitim başlatılmaz. Toplam girdi sayısının SUCCESS + SKIPPED + ERROR toplamına eşit olması, sample_id değerlerinin benzersiz olması ve SUCCESS durumundaki dosyaların diskte gerçekten bulunması kontrol edilir.

Kontrol	Gerçek çalıştırmadaki değer
Toplam girdi	3000
Başarılı ROI	1962
Atlanan	1038
Hata	0
Eğitime uygun başarılı örnek	1962
Benzersiz kaynak video	1429

5.1 Split sızıntısı neden kritik?

Aynı videodan gelen birbirine çok benzeyen frame'ler train ve test kümelerine aynı anda düşerse model gerçek bir deepfake genellemesi yapmak yerine videoyu veya kişiyi ezberleyebilir. Bu nedenle notebook train, validation ve test kümelerindeki source_video kümelerini karşılaştırır ve herhangi bir kesişim varsa eğitimi durdurur.

Kesişim	Video sayısı
Train ↔ Validation	0
Train ↔ Test	0
Validation ↔ Test	0

Gerçek çalıştırmada üç kesişim de 0 çıkmıştır. Bu, kullanılan metadata düzeyinde aynı kaynak videonun birden fazla split'e dağılmadığını gösterir.

6. Görüntü Ön İşleme

Her başarılı kaş ROI görüntüsü OpenCV ile gri tona çevrilmiş olarak okunur ve 128×64 piksele yeniden boyutlandırılır. Bu işlem Drive'daki orijinal kaş görüntüsünü değiştirmez; yalnızca o an RAM'de oluşturulan kopya üzerinde uygulanır.

- Grayscale: Renk bilgisini kaldırır ve yapısal/dokusal bilgiyi öne çıkarır.
- 128×64 resize: Bütün örnekleri HOG ve LBP için aynı geometrik boyuta getirir.
- uint8 kontrolü: Piksel veri tipini tutarlı hale getirir.

7. HOG Özellik Çıkarımı

HOG (Histogram of Oriented Gradients), görüntüdeki parlaklık değişimlerinin yönlerini ve büyüklüklerini özetleyen bir özellik çıkarım yöntemidir. Kaş bölgesi açısından bu özellik; kaş çizgisinin eğimi, kenar yapısı, kıl yönelimleri ve genel geometrik form gibi bilgileri temsil etmeye yardımcı olur.

- 9 gradient yönelimi kullanılır.
- Görüntü 8×8 piksellik hücreler halinde değerlendirilir.

- Komşu 2x2 hücre blokları birlikte normalize edilir.
- L2-Hys normalizasyonu aydınlık/kontrast değişimlerine karşı daha kararlı temsil sağlar.

Gerçek çalıştırmada her görüntü için HOG vektörü 3780 boyutludur.

8. LBP Özellik Çıkarımı

LBP (Local Binary Pattern), bir pikselin çevresindeki komşularla olan yerel yoğunluk ilişkisini kodlayan bir doku tanımlayıcısıdır. Bu deneyde amaç, kaş kılları, deri yüzeyi ve kaş çevresindeki mikro-doku örüntülerinin Real/Fake ayırımına katkı sağlayıp sağlamadığını ölçmektir.

- 8 komşu nokta kullanılır.
- Yarıçap 1 pikseldir.
- 'uniform' LBP yöntemi kullanılır.
- LBP görüntüsü doğrudan SVM'e verilmez; normalize edilmiş histogram üretilir.

Gerçek çalıştırmada LBP histogramı 10 boyutludur.

9. HOG ve LBP Özellik Füzyonu

Her görüntü için HOG ve LBP ayrı ayrı çıkarıldıktan sonra iki vektör uç uca eklenir (concatenation). Böylece SVM hem şekil/yönelim hem de doku bilgisini aynı anda görür.

Özellik	Boyut
HOG	3780
LBP	10
Birleştirilmiş özellik vektörü	3790

Notebook ayrıca NaN veya Inf içeren özellik vektörü olup olmadığını kontrol eder. Sayısal bozukluk tespit edilirse süreç sessizce devam etmez.

10. Smoke Test ve Özellik Cache Mekanizması

Tüm veri üzerinde uzun feature extraction başlamadan önce train setinden iki Real ve iki Fake örnek seçilir. Bu dört örnek üzerinde HOG+LBP çıkarımı, StandardScaler fit'i, küçük bir Linear SVM eğitimi ve tahmin işlemi çalıştırılır. Bu klasik-ML smoke test başarısızsa ana deney devam etmez.

Daha sonra train, validation ve test için özellikler ayrı ayrı çıkarılır ve sıkıştırılmış NPZ dosyalarına atomik olarak kaydedilir. Aynı run yeniden başlatılırsa mevcut feature cache dosyaları yeniden hesaplama yerine yüklenebilir. Bu, özellikle Google Drive üzerindeki binlerce ROI için ciddi zaman kazandırır.

11. StandardScaler: Neden Sadece Train Üzerinde Öğreniyor?

HOG ve LBP özelliklerinin ölçekleri aynı değildir. Bu nedenle birleştirilmiş özellik vektörleri StandardScaler ile standartlaştırılır. Ancak scaler'ın ortalama ve standart sapma değerleri yalnızca train setinden öğrenilir. Validation ve test yalnızca train'den öğrenilen scaler ile dönüştürülür.

Bu ayırım veri sızıntısını önler. Validation veya test istatistiklerinin scaler eğitimine dahil edilmesi, modelin görmemesi gereken gelecekteki bilgiye dolaylı erişmesi anlamına gelirdi.

12. SVM Eğitimi ve Model Seçimi

Notebook tek bir SVM ayarı seçip doğrudan test etmez. Önceden tanımlanmış adaylar train setinde eğitilir ve yalnızca validation setindeki performanslarına göre karşılaştırılır.

Linear kernel için üç C değeri; RBF kernel için üç C ve üç gamma seçeneği denenir. Böylece toplam 12 aday değerlendirilmiştir.

Kernel	C adayları	Gamma adayları
Linear	C = 0.1, 1.0, 10.0	Gamma kullanılmaz
RBF	C = 0.1, 1.0, 10.0	scale, 0.001, 0.0001

12.1 Karar threshold'u neden ayrıca seçiliyor?

SVC, her örnek için bir decision score üretir. Varsayılan olarak sıfır sınırı kullanılabilir; ancak bu notebook validation setinde Precision-Recall eğrisi üzerinden F1 değerini en yüksek yapan threshold'u bulur. Bu threshold daha sonra sabitlenir ve test setinde değiştirilmez.

Seçilen ayar	Değer
Seçilen kernel	linear
C	0.1
Gamma	None
Threshold	-1.355764
Seçim seti	validation
Seçim metriği	f1

Gerçek çalıştırmada en iyi aday Linear SVM, C=0.1 olmuştur. Model ve threshold validation F1'e göre seçilmiş; test seti model seçimine dahil edilmemiştir.

13. Checkpoint ve Tekrar Kullanılabilir Model Kaydı

Bu deney PyTorch tabanlı bir sinir ağı olmadığı için epoch, optimizer_state_dict veya forward/backward gibi alanlar doğal olarak yoktur. Notebook bu alanları varmış gibi üretmek yerine 'klasik makine öğrenmesine uygulanamaz' şeklinde açıkça işaretler.

- best.ckpt: seçilen SVM, scaler, threshold, config ve RNG durumlarını içerir.
- last.ckpt: aynı klasik-ML durumunun son kayıt noktasıdır.
- checkpoint_quality_gate.ckpt: atomik kayıt/yükleme doğrulaması için kullanılır.
- svm_model.joblib: bağımsız inference için model + scaler + threshold paketidir.

Checkpoint geçici bir dosyaya yazılır, tekrar yüklenip gerekli anahtarlar doğrulanır ve ancak ondan sonra hedef dosyanın yerine atomik olarak geçirilir. Ardından yeniden yüklenen modelin validation skorlarının orijinal modelle aynı olduğu kontrol edilir.

14. Nihai Test Değerlendirmesi

Test seti, model ve threshold seçimi tamamlandıktan sonra ilk kez değerlendirilir. Her test frame'i için decision score, gerçek sınıf, tahmin edilen sınıf ve tahminin doğru olup olmadığı kaydedilir.

14.1 Frame-level değerlendirme

Her kaş görüntüsü bağımsız bir örnek kabul edilir ve performans frame bazında hesaplanır.

14.2 Video-level değerlendirme

Aynı source_video'ya ait test framelerinin SVM decision score'ları ortalananır. Ortalama skor sabit threshold ile karşılaştırılarak video için tek bir Real/Fake tahmini üretilir. Böylece frame düzeyindeki küçük oynamaların video kararına etkisi azaltılmaya çalışılır.

15. Hesaplanan Performans Metrikleri Ne Anlama Geliyor?

Metrik	Anlamı
Accuracy	Tüm örnekler içinde doğru sınıflandırılanların oranı.
Precision	Fake denilen örneklerin ne kadarının gerçekten Fake olduğu.
Recall / Sensitivity	Gerçek Fake örneklerin ne kadarının yakalandığı.
F1-score	Precision ve Recall'un harmonik ortalaması.
Specificity	Gerçek Real örneklerin ne kadarının doğru biçimde Real olarak tanındığı.
ROC-AUC	Model skorlarının iki sınıfı eşikten bağımsız ayırma gücü.
Average Precision	Precision-Recall eğrisi üzerinden özellikle pozitif sınıf ayırma kalitesini özetler.

16. Gerçek Çalıştırmada Elde Edilen Sonuçlar

Aşağıdaki değerler notebook'un tamamlanmış gerçek run_summary çıktısından alınmıştır. Bunlar kodun nasıl çalıştığını açıklamak yanında, bu HOG+LBP+SVM baseline'inin mevcut veri üzerindeki performansını da göstermektedir.

Seviye	Accuracy	Precision	Recall	F1	Specificity	ROC-AUC	Avg. Precision
Frame	%50.51	%49.40	%86.32	%62.84	%16.83	0.540	0.513
Video	%57.25	%59.32	%86.42	%70.35	%15.79	0.553	0.657

16.1 Sonuçların yorumu

Model Fake sınıfını yakalamada yüksek Recall üretmiştir; frame düzeyinde yaklaşık %86.32, video düzeyinde yaklaşık %86.42 Fake yakalama oranı vardır. Buna karşılık Specificity oldukça düşüktür. Bu, çok sayıda gerçek örneğin yanlış biçimde Fake olarak işaretlendiğini gösterir.

ROC-AUC değerlerinin frame düzeyinde yaklaşık 0.540 ve video düzeyinde yaklaşık 0.553 olması, HOG+LBP özelliklerinin bu veri üzerinde iki sınıfı güçlü biçimde ayırmadığını göstermektedir. Video-level birleştirme Accuracy ve F1'i yükseltmiş olsa da ayırıştırma gücü halen sınırlıdır.

Bu sonuç, deneyin başarısız olduğu anlamına gelmez. Bu model bir baseline'dır ve önemli bilimsel soru şudur: 'Kaş bölgesinden yalnız HOG+LBP gibi klasik özellikler kullanıldığında ne kadar ayırım elde edilebiliyor?' Mevcut run bu soruya ölçülebilir bir referans sağlar.

17. Quality Gate Kontrolleri

Kontrol	Durum
Metadata schema	PASS
Split leakage	PASS
Data accounting	PASS
NaN/Inf feature	PASS
Classical ML smoke test	PASS
PyTorch forward/backward	NOT_APPLICABLE_CLASSICAL_ML
Checkpoint save/reload	PASS
Fresh-load inference	PASS
Test model seçiminde kullanıldı mı?	False

PyTorch forward/backward testinin 'NOT_APPLICABLE_CLASSICAL_ML' olması hata değildir. HOG+LBP+sklearn SVC, PyTorch eğitim döngüsü kullanmadığı için bu kontrol mantıksal olarak uygulanamaz.

18. Üretilen Grafikler

Notebook gerçek çalıştırmada 9 adet rapor grafiği üretmiştir. Grafikler İngilizce başlık/eksenlerle ve minimum kısa kenar çözünürlük kontrolüyle kaydedilir.

- Dataset distribution

- Validation F1 by SVM candidate
- Frame-level confusion matrix
- Video-level confusion matrix
- Frame-level ROC curve
- Video-level ROC curve
- Frame-level Precision-Recall curve
- Video-level Precision-Recall curve
- Frame-level SVM decision score distribution

19. Sonuç Klasörüne Kaydedilen Çıktılar

Her çalışma benzersiz bir Run ID altında tutulur. Böylece eski deney sonuçları bozulmaz ve farklı konfigürasyonlarla yapılan deneyler birbirinden ayrılabilir.

```
<run_id>/
├── checkpoints/
│   ├── best.ckpt
│   ├── last.ckpt
│   └── checkpoint_quality_gate.ckpt
├── logs/
│   └── run.log
├── metrics/
│   ├── data_accounting.json
│   ├── svm_validation_search.csv
│   ├── svm_selection.json
│   ├── quality_gates.json
│   ├── test_metrics_frame_level.json
│   ├── test_metrics_video_level.json
│   └── test_metrics_summary.csv
├── predictions/
│   ├── test_predictions_frame_level.csv
│   └── test_predictions_video_level.csv
├── figures/
├── artifacts/
│   ├── metadata_used.csv
│   ├── features_train.npz
│   ├── features_val.npz
│   ├── features_test.npz
│   ├── feature_dimensions.json
│   ├── feature_pipeline.json
│   ├── feature_preprocessing.joblib
│   └── svm_model.joblib
├── config_resolved.yaml
├── requirements_lock.txt
├── environment.json
├── output_manifest.csv
└── run_summary.json
```

20. Notebook'un Yapmadığı Şeyler

- Kaş görüntülerini yeniden kırpmaz; giriş olarak hazır Eyebrow ROI görüntülerini kullanır.
- Drive'daki kaş frame dosyalarını değiştirmez, silmez, taşımaz veya yeniden adlandırmaz.
- Frame düzeyinde rastgele yeni train/validation/test ayrımı yapmaz; mevcut splitleri korur.
- Pretrained ResNet, VGG, EfficientNet, Swin veya başka bir CNN kullanmaz.

- HOG veya LBP'yi 'eğitmez'; bunlar deterministik özellik çıkarım yöntemleridir.
- Model seçimi için test setine bakmaz.
- PyTorch optimizier/epoch mantığını SVM'e yapay biçimde uygulamaz.

21. Genel Sonuç

Bu notebook, kaş bölgesinden deepfake tespiti için temiz ve izlenebilir bir klasik makine öğrenmesi baseline'ı oluşturur. Kaş ROI'lerinden HOG ile şekil/yönelim, LBP ile mikro-doku özellikleri çıkarılır; bu özellikler birleştirilir, train verisine göre standartlaştırılır ve SVM ile Real/Fake sınıflandırması öğrenilir. Model ve threshold yalnızca validation seti üzerinden seçilir; test seti nihai değerlendirmeye saklanır. Aynı zamanda metadata, video sızıntısı, veri muhasebesi, sayısal değerler, checkpoint ve yeniden yükleme testleriyle deneyin tekrar üretilebilir ve denetlenebilir olması hedeflenir.

Gerçek run, Linear SVM ($C=0.1$) seçmiş ve video seviyesinde F1 yaklaşık %70.35 üretmiştir; ancak düşük specificity ve yaklaşık 0.55 ROC-AUC değeri, HOG+LBP özelliklerinin mevcut kaş verisi üzerinde tek başına sınırlı ayırıştırma gücüne sahip olduğunu göstermektedir. Bu nedenle deney, sonraki modellerin veya daha zengin özelliklerin karşılaştırılacağı anlamlı bir baseline olarak değerlendirilmelidir.

22. Raporun Dayandığı Dosyalar

- Düzeltilmiş HOG_LBP_SVM_Eyebrow_Deepfake_Colab notebook'u
- ML Proje & Kod Standardı — Deepfake Detection, v2.0
- Tamamlanmış run_summary.json — 20260807_0948_eyebrow_hog_lbp_svm_seed42
- feature_pipeline.json ve config_resolved.yaml